Programmazione 2

a.a. 2023-2024

Puntatori a funzione e puntatori a void

Puntatori a funzione

Puntatori a funzione

- Il nome di una funzione è in realtà l'indirizzo iniziale in memoria del codice che esegue il compito della funzione.
- Un **puntatore a una funzione** contiene l'**indirizzo** della funzione in memoria.
- I puntatori alle funzioni possono essere:
  - memorizzati in variabili del tipo opportuno,
  - passati a funzioni,
  - restituiti da funzioni, e
  - confrontati tra loro per uguaglianza o disuguaglianza.

Puntatori a funzione

- ★ Un puntatore a funzione è un puntatore che punta all'indirizzo di memoria in cui è contenuto il codice eseguibile di una funzione.
- ★ La memoria indirizzata da un puntatore a funzione si trova nel text segment di memoria associata al processo (memoria statica).
- ★ La memoria indirizzata da un puntatore a funzione non deve, quindi, essere allocata
- ★ Importante: un puntatore a funzione punta sempre a una funzione con una specifica segnatura
- ★ Lo stesso puntatore può indirizzare funzioni diverse ma con gli stessi (tipi di) parametri e lo stesso tipo di valore di ritorno

Puntatori a funzione: esempio

- Prototipo di funzione che riceve un intero e restituisce un intero:

```
int miaFunz (int);
```

- Dichiarazione di un puntatore a funzione che riceve un intero e restituisce un intero:

```
int (* miaFunzPtr)(int);
```

- Definizione del nuovo tipo MiaFunzPtr per puntatori a funzione che riceve un intero e restituisce un intero:

```
typedef int (* MiaFunzPtr)(int);
```

- Dichiarazione di una variabile di tipo MiaFunzPtr

```
MiaFunzPtr funz;
```

Puntatori a funzione: esempio

Supponiamo di voler scrivere una funzione che esegue una generica operazione su tutti gli elementi di un array di interi.

```
typedef int (*FN_op)(int);

int square(int a) { return (a * a);}

int increment(int a) { return (a + 1);}

int double(int a) { return (2 * a);}

Array_Op(int *A, int dim, FN_op op) {
  for (i=0; i < dim; i++)  A[i] = op(A[i]);
}

int main(void) {

   Array_op(A,n,&square);
   Array_op(A,n,&increment);
   Array_op(A,n,&double);
}
```

Puntatori a funzione

I puntatori a funzione:

- ★ Risultano particolarmente utili per rendere il codice più generale, disaccoppiando la funzione chiamante dalla funzione chiamata.
- ★ Agevolano il riuso del codice
- ★ Sono molto utili per l'implementazione di Tipi di Dati Astratti (ADT, Abstract Data Types), unitamente ai puntatori a void (void *)

Puntatori a void

Puntatori a void

Spesso è utile dichiarare variabili che possano contenere tipi di dati diversi. In C un tipo di dato generico si chiama void

Il C non ammette, però, variabili di tipo generico void:

```
void a;    /* dichiarazione non valida */
```

Ammette, invece, puntatori a dati di tipo generico void:

```
void* p;    /* dichiarazione valida */
```

Un puntatore void può essere usato per indirizzare qualsiasi tipo di dato

```
int x; void* p = &x;     /*  p punta a un int  */
float f; void* p = &f;   /*  p punta a un float */
```

Nota: Il C tratta tutti i puntatori a void come puntatori a singoli byte, quindi:

se: `int* x; e void* p = x;` allora `(p+i) ≠ (x+i)` !

De-referenziazione di puntatori a void

I puntatori a void non possono essere de-referenziati direttamente

- `void* p; printf("%d", *p);  /* non valido */`

Devono essere sempre sottoposti a cast prima di accedere al loro contenuto:

- `void* p; printf("%d",*((int*)p); /* valido */`

Analogamente, l'assegnamento di un valore al contenuto di un puntatore a void necessita di cast:

- `int x; void* p = &x;  *p = 5;  /* non valido */`
- `intx;  void* p = &x;  *((int*)p) = 5; /* valido */`

In alternativa, è possibile usare la funzione di libreria memcpy(dest,source,n_byte) per copiare un numero dato di byte contigui da un qualsiasi indirizzo sorgente a un indirizzo destinazione, indipendentemente dal tipo di dati contenuti:

- `int x; int y = 5; void* p = &x;`

  `memcpy(p,&y,sizeof(int)); /* valido */`

Puntatori a funzione e puntatori a void: esempio

La combinazione di puntatori a funzione e puntatori a void rende possibile la scrittura di codice generico in C.

Esempio: Si supponga di voler generalizzare l'algoritmo di ordinamento SelectionSort per ordinare array che contengono un generico tipo di dati.

- Tipi di dati diversi hanno, infatti, diverse logiche di confronto (ad es. interi, stringhe, ecc.).
- Possiamo definire SelectionSort in modo che utilizzi una callback compare() per confrontare gli elementi del generico array che deve ordinare.
- compare() punterà a diverse funzioni di confronto, a seconda del tipo di dati correntemente contenuti nell'array

Ordinamento di array generici

```
typedef int (*COMPAREFN)(void *,void *);

int ConfrontaInt(void *a, void *b) {
   int inta = *((int *)a);
   int intb = *((int *)b);
   return ((inta > intb)? 1 :((inta < intb)? -1 : 0));
}
```

```
void SelectionSort(void* base, int n, int elemsize, COMPAREFN compare);

int arr[] = {20, 15, 36, -8, 2, 7};

SelectionSort(arr, 6, sizeof(int), &ConfrontaInt);
```